

Travaux Pratiques

Requêtes SQLAlchemy 2 et intégration Flask Module : Développement Web avec Flask

Formation DSBD | Année 2025-2026

Objectifs pédagogiques

- Maîtriser la syntaxe **moderne de SQLAlchemy 2** (`select`, `Mapped`, `mapped_column`).
- Effectuer des requêtes de lecture, filtrage, tri, jointure et agrégation.
- Insérer, modifier et supprimer des données via la session SQLAlchemy.
- Exposer les données via des **routes Flask** et les afficher dans des **templates Jinja2**.
- Comprendre les relations entre modèles (1-1, 1-N, N-N) et les interroger.

Durée estimée : 3h30 **Support :** documents autorisés **Base de données :** SQLite (`instance/project.db`)

Toutes les réponses doivent utiliser la syntaxe **SQLAlchemy 2** (`db.session.execute(select(...))`).
L'ancienne syntaxe `Model.query` est **interdite**.

1. Contexte du projet

Le projet Flask sur lequel vous travaillez modélise une plateforme pédagogique simple. Voici un rappel du schéma de données implémenté dans `models.py` :

Modèle	Table	Description
User	user	Compte utilisateur (login, mot de passe, actif)
Profile	profile	Informations personnelles liées à un User (1-1)
Homework	homework	Devoir avec titre, description, date limite
Session	session	Session d'examen (type : NORMALE / RATTRAPAGE)
	session_homework	Table d'association Session ↔ Homework (N-N)

Rappel syntaxe SQLAlchemy 2 :

```

1 from sqlalchemy import select, func, and_, or_, desc
2 from models import User, Profile, Homework, Session
3
4 # Lecture de tous les utilisateurs
5 stmt = select(User)
6 users = db.session.scalars(stmt).all()
7
8 # Lecture d'un seul enregistrement
9 user = db.session.scalar(select(User).where(User.id == 1))

```

2. Partie I – Préparation et peuplement de la base

2.1. Exercice 1 – Création de la base et migration

1. Vérifiez que l'environnement virtuel est activé et que les dépendances listées dans `requirements.txt` sont bien installées.
2. Initialisez les migrations si ce n'est pas encore fait, puis appliquez-les :

```
flask db init      # une seule fois
flask db migrate -m "init"
flask db upgrade
```

3. Confirmez la création des tables en ouvrant `instance/project.db` avec un outil SQLite (ex. : *DB Browser for SQLite* ou l'extension VS Code *SQLite Viewer*).
4. **Question** : expliquez en deux phrases la différence entre `flask db migrate` et `flask db upgrade`.

2.2. Exercice 2 – Peuplement via un script

Créez un fichier `seed.py` à la racine du projet. Ce script doit :

1. Importer `app` et `db` depuis `app.py` et `extensions/sqlalchemy.py`.
2. Dans un contexte applicatif (`with app.app_context()`), insérer les données suivantes en utilisant **uniquement** la session SQLAlchemy (`db.session.add, db.session.commit`) :
 - 5 utilisateurs (au moins 2 inactifs, i.e. `is_active=False`).
 - 5 profils associés, dont au moins 2 de sexe F et 3 de sexe M, avec des dates de naissance variées (entre 1990 et 2002).
 - 4 devoirs (**Homework**) avec des dates limites différentes (dont 1 déjà dépassée).
 - 2 sessions d'examen : une **NORMALE** et une **RATTRAPAGE**, chacune associée à au moins 2 devoirs.
 - Associer un correcteur (`corrected_by`) à au moins 2 devoirs.
3. Vérifiez l'insertion en affichant dans le terminal le nombre d'enregistrements dans chaque table.

Structure attendue de `seed.py` :

```
1 import json
2 from app import app
3 from extensions.sqlalchemy import db
4 from models import User, Profile, Homework, Session, SexeType,
   SessionType
5 from datetime import datetime, timezone, date
6
7 with app.app_context():
8     # -- Nettoyage (optionnel, pour re-seeder proprement)
9     -----
10    db.session.execute(db.delete(Session))
11    db.session.execute(db.delete(Homework))
12    db.session.execute(db.delete(Profile))
13    db.session.execute(db.delete(User))
14    db.session.commit()
15
16    # -- Utilisateurs
17    -----
```

```

16 u1 = User(username="alice", password="hash1", is_active=True)
17 u2 = User(username="bob", password="hash2", is_active=True)
18 u3 = User(username="charlie", password="hash3", is_active=False)
19 u4 = User(username="diana", password="hash4", is_active=True)
20 u5 = User(username="eve", password="hash5", is_active=False)
21 db.session.add_all([u1, u2, u3, u4, u5])
22 db.session.flush() # obtenir les id sans commit
23
24 # -- Profils
25 # -----
26 # A COMPLETER ...
27
28 # -- Devoirs
29 # -----
30 # A COMPLETER ...
31
32 # -- Sessions
33 # -----
34 # A COMPLETER ...
35
36 db.session.commit()
37 print("Base peupl e avec succ s.")

```

3. Partie II– Requêtes SQLAlchemy 2 « standalone »

Créez un fichier `queries.py` à la racine du projet. Chaque question doit être résolue dans une fonction dédiée appelée dans un bloc `with app.app_context()`.

Rappel des imports utiles :

```

from sqlalchemy import select, func, and_, or_, not_, desc, asc
from datetime import datetime, timezone

```

3.1. Requêtes de lecture simple

- Q1. Tous les utilisateurs.** Récupérez et affichez la liste complète des utilisateurs (`id`, `username`, `is_active`).
- Q2. Utilisateur par username.** Écrivez une fonction `get_user_by_username(username)` qui retourne l'utilisateur correspondant, ou `None` si inexistant. Utilisez `scalar()`.
- Q3. Utilisateurs actifs.** Récupérez uniquement les utilisateurs dont `is_active` vaut `True`, triés par `username` alphabétiquement (`order_by`).
- Q4. Comptage.** Affichez le nombre total d'utilisateurs actifs en utilisant `func.count()`.
- Q5. Premier et dernier.** Récupérez l'utilisateur créé en premier et celui créé en dernier (`created_at`), en utilisant `asc` et `desc`.

3.2. Filtrage avancé

- Q6. Recherche partielle.** Récupérez tous les utilisateurs dont le **username** contient la lettre « a » (insensible à la casse) en utilisant **ilike**.
- Q7. Combinaison de conditions.** Récupérez tous les utilisateurs actifs **ET** dont le **username** commence par « a » ou « b ». Utilisez **and_** et **or_**.
- Q8. Exclusion.** Récupérez tous les profils dont le sexe **n'est pas SexeType.M**. Utilisez **not_** ou l'opérateur **!=**.
- Q9. Filtrage sur date.** Récupérez les devoirs dont la **due_date** est **déjà dépassée** par rapport à la date/heure actuelle.
- Q10. Plage de valeurs.** Récupérez les profils dont la date de naissance (**date_of_birth**) est comprise entre le 1^{er} janvier 1995 et le 31 décembre 2000 inclus. Utilisez **between**.

3.3. Jointures et relations

- Q11. Jointure interne (1-1).** Listez chaque utilisateur avec son email et son nom complet (prénom + nom) en effectuant une jointure entre **User** et **Profile**.
- Q12. Jointure externe (LEFT JOIN).** Listez **tous** les utilisateurs avec leur email de profil s'il existe, **NULL** sinon. Utilisez **outerjoin**.
- Q13. Relation Many-to-Many.** Pour chaque **Session**, affichez son type et la liste des titres des devoirs qui lui sont associés. Naviguez via l'attribut de relation **session.homeworks**.
- Q14. Devoirs avec correcteur.** Récupérez tous les devoirs qui ont un correcteur assigné (**corrected_by** non nul) et affichez le titre du devoir ainsi que le username du correcteur. Effectuez une jointure explicite.

3.4. Agrégations et sous-requêtes

- Q15. Comptage par groupe.** Comptez le nombre de profils par sexe (**SexeType**) en utilisant **func.count** et **group_by**.
- Q16. Devoir le plus récent.** Récupérez le devoir dont **created_at** est le plus récent en utilisant **func.max** dans une sous-requête ou via **order_by + limit**. Proposez les deux approches.

3.5. Opérations d'écriture

- Q17. Insertion.** Insérez un nouvel utilisateur avec son profil en une seule transaction. Vérifiez l'insertion par une requête **select** immédiate.
- Q18. Mise à jour.** Désactivez tous les utilisateurs dont le **username** commence par « e » en utilisant **db.session.execute(db.update(...).where(...).values(...))**.
- Q19. Suppression.** Supprimez le profil de l'utilisateur « eve » puis l'utilisateur lui-même. Respectez l'ordre de suppression pour éviter les erreurs de contraintes de clé étrangère.

4. Partie III – Intégration Flask : routes, vues et templates

Dans cette partie, vous allez créer de nouvelles routes dans **app.py** et les templates Jinja2 associés dans le répertoire **templates/**.

Rappel : tous les templates doivent hériter de `base.html` via `{% extends 'base.html' %}`. La navigation dans `base.html` devra être mise à jour pour inclure les nouveaux liens.

4.1. Liste des utilisateurs

Route : GET `/users`

1. Créez la route `/users` dans `app.py`.
2. La vue doit récupérer **uniquement les utilisateurs actifs** triés par **username**, en utilisant SQLAlchemy 2.
3. Créez le template `templates/users.html` qui affiche :
 - Un tableau HTML avec les colonnes : *ID*, *Username*, *Statut*, *Membre depuis*.
 - La date **created_at** formatée avec le filtre Jinja2 **strftime** ou directement via `.strftime('%d/%m/%Y')`.
 - Un badge coloré « Actif » / « Inactif » selon la valeur de **is_active**.
 - Un lien « Voir le profil » vers la route du sous-exercice suivant.
 - Un message « Aucun utilisateur trouvé » si la liste est vide (utilisez `{% if %}...{% else %}`).
4. Ajoutez un lien « Utilisateurs » dans la barre de navigation de `base.html`.

4.2. Profil d'un utilisateur

Route : GET `/users/<int:user_id>`

1. Créez la route dynamique `/users/<int:user_id>`.
2. Récupérez l'utilisateur par son **id**. Si l'utilisateur n'existe pas, retournez une page d'erreur 404 en utilisant `abort(404)`.
3. Créez le template `templates/user_detail.html` qui affiche :
 - Le **username** et le statut de l'utilisateur.
 - Les informations du profil : prénom, nom, email, téléphone, sexe, date de naissance.
 - Si l'utilisateur n'a pas de profil, affichez « Profil non renseigné ».
 - Un bouton « Retour à la liste » pointant vers `/users`.

4.3. Liste des devoirs

Route : GET `/homeworks`

1. Créez la route `/homeworks` qui récupère tous les devoirs triés par **due_date** croissante.
2. Créez le template `templates/homeworks.html` qui affiche :
 - Un tableau avec : *Titre*, *Description*, *Date limite*, *Correcteur*, *Statut*.
 - Pour la colonne *Statut* : affichez « En retard » si la **due_date** est passée, sinon « À venir ». Comparez dans le template en passant `now=datetime.now(timezone.utc)` depuis la vue.

- Pour la colonne *Correcteur* : le username du correcteur ou « Non assigné ».
- Le **nombre total de devoirs** affiché au-dessus du tableau, en utilisant **loop.length** ou **|length** dans Jinja2.

4.4. Recherche d'utilisateurs

Route : GET `/users/search`

1. Créez la route `/users/search`.
2. La vue lit le paramètre de requête **q** (`request.args.get('q', '')`) et effectue une recherche **ilike** sur **username** et sur **Profile.nom** (jointure nécessaire).
3. Si **q** est vide, retournez tous les utilisateurs actifs.
4. Créez le template `templates/search.html` avec :
 - Un formulaire `<form method="GET" action="/users/search">` contenant un `<input>` de recherche et un bouton.
 - La valeur saisie pré-remplie dans l'input (attribut **value**).
 - Les résultats affichés dans un tableau identique à celui de `/users`.
 - Le terme recherché mis en évidence dans un message : « X résultat(s) pour « q » ».

4.5. Tableau de bord des sessions

Route : GET `/sessions`

1. Créez la route `/sessions` qui récupère toutes les sessions d'examen.
2. Pour chaque session, calculez dans la vue le nombre de devoirs associés.
3. Créez le template `templates/sessions.html` qui affiche :
 - Deux sections distinctes : « Sessions Normales » et « Sessions de Rattrapage », en utilisant `{% if session.session_type.value == 'NORMALE' %}` ou en filtrant depuis la vue.
 - Pour chaque session : son *id*, sa *date de création* et la liste des titres des devoirs associés (balise `<ul>`).
 - Si une section est vide, affichez « Aucune session de ce type ».
 - Un résumé en bas de page : nombre total de sessions, nombre de sessions normales, nombre de sessions de rattrapage.
4. **Question** : pourquoi est-il préférable d'effectuer les calculs dans la vue Python plutôt que dans le template Jinja2 ? Répondez en deux phrases dans un commentaire dans `app.py`.

5. Partie IV – Bonus et approfondissement

Bonus 1 – Pagination

Modifiez la route `/users` pour supporter la pagination :

- Paramètre GET : **page** (défaut : 1), **per_page** (défaut : 3).

- Utilisez **limit** et **offset** dans la requête SQLAlchemy.
- Affichez des liens « Page précédente » / « Page suivante » dans le template.
- Calculez le nombre total de pages via **func.count** et **math.ceil**.

Bonus 2 – Statistiques

Créez la route **GET /stats** et le template [templates/stats.html](#) affichant :

- Nombre d'utilisateurs actifs / inactifs.
- Répartition H/F des profils (nombre et pourcentage).
- Devoir avec la date limite la plus proche (non dépassée).
- Nombre moyen de devoirs par session (**func.avg** ou calcul Python).
- Utilisateur inscrit le plus récemment.

Toutes ces statistiques doivent être calculées par des requêtes SQLAlchemy 2 distinctes.

Bonus 3 – Gestion des erreurs

1. Créez un gestionnaire d'erreur 404 avec **@app.errorhandler(404)** et le template [templates/404.html](#).
2. Créez un gestionnaire d'erreur 500 ([templates/500.html](#)).
3. Documentez dans [app.py](#) les cas d'utilisation de **abort()** dans les routes que vous avez créées.

6. Ressources

- Documentation SQLAlchemy 2 : <https://docs.sqlalchemy.org/en/20/>
- Documentation Flask : <https://flask.palletsprojects.com/>
- Documentation Flask-SQLAlchemy : <https://flask-sqlalchemy.readthedocs.io/>
- Documentation Jinja2 : <https://jinja.palletsprojects.com/>
- [models.py](#), [app.py](#), [extensions/](#) du projet courant.

Fichiers attendus en rendu : [seed.py](#), [queries.py](#), [app.py](#) (modifié), tous les templates créés.